

Exploring CNN, Autoencoders, VAE and GAN for Fashion-MNIST

Arne Elias Tidemand Ruud

University of Oslo

(Dated: June 15, 2025)

This report explores several deep learning models on the basic and commonly used Fashion-MNIST dataset. Start with a basic convolutional neural network (CNN) for image classification. It reaches 86.7% test accuracy after tuning learning rate and regularization. Then i try to test three generative models: a convolutional autoencoder (AE), a variational autoencoder (VAE), and a generative adversarial network (GAN). I mainly try to reconstruct input images and then compare them. The VAE produces smoother results but less detailed textures. The best VAE model without dropout achieves a validation loss of 0.2591. For each model, we perform grid search over learning rate and training epochs. We plot loss curves to find stable training regions and select the best parameters. I had worse time with GAN model. It initially produced checkerboard artifacts. Evn after fixing this training is unstable and produces noisy images hard to interpret anything of. We compare reconstruction quality, loss curves, and training stability across all models. For the future i wanna fix the GAN model, visualize latent space and produce novel new samples of images to then use CNN to see if it can classify them.

CONTENTS

I. Introduction	2
II. The dataset - Fashion-MNIST	2
III. Method	2
A. Shared concepts and setup	2
B. Convolutional Neural Network (CNN)	3
C. Autoencoder (AE)	4
D. Variational Autoencoder (VAE)	5
E. Generative Adversarial Network (GAN)	5
IV. Results	6
A. CNN	6
B. Autoencoder	7
C. Variational Autoencoder	8
D. GAN Results	9
V. Discussion	10
A. CNN	10
B. Autoencoder	10
C. Variational Autoencoder	11
D. GAN	11
VI. Conclusion	11
Future work and improvements	11
References	13

I. INTRODUCTION

In this report, i look at four different neural network models and how they perform on the Fashion-MNIST dataset. The idea is to explore how each model learns from the data and what kind of results they give. Fashion-MNIST is a simple dataset with grayscale images of clothes like shoes, shirts, and bags. Its not too big or complex, which makes it a good fit for testing and comparing different types of models and allowed for easy modifications of code to be tested within a limited time frame. i use TensorFlow with the Keras API for all implementations. Its a high-level framework lets me do building, training, and testing neural networks easier but allows custom losses and layers if thats desirable. I make use of good amount of existing code from either lecture notes or existing code bases.

The models covered are a Convolutional Neural Network (CNN), a basic Autoencoder (AE), a Variational Autoencoder (VAE), and a Generative Adversarial Network (GAN). The CNN is used for supervised classification, while the other three are unsupervised models trained to reconstruct or generate new images. These models differ in how they learn features and represent data internally. Some rely on reconstruction, others on learned distributions or adversarial feedback.

Since they are different methods and used to do different things, im not really trying to find which one is “best,” but more to understand how they work. What kind of outputs they produce and what kind of structure they learn from the data. For each model i go through some basic theory, explain how i set it up, and show some results. Its worth mentioning that the results are kept relatively limited. With more time, one could explore things like traversals through the latent space, interpolation between samples, plotting the learned latent clusters for VAEs, or even using the discriminators output for anomaly detection. These would give a deeper look into what the models are actually learning.

II. THE DATASET - FASHION-MNIST

This project made use of the commonly used dataset named Fashion-MNIST dataset. Widely used for benchmarking for image classification and generative models it provides an easy to use dataset to test out different methods. Ideally i wanted to try out “The Quick, Draw! Dataset” from the google game where users do handdrawn images of different objects, but i was worried about the training time considering the time restraints as the images is very sparse and vary heavily. Image below shows some images from the set(with predictions using CNN.)



Figure 1. Example images from the Fashion-MNIST dataset with predicted and true labels. Green text indicates correct predictions, red text indicates incorrect ones. Source: <https://blog.tensorflow.org/2018/04/fashion-mnist-with-tfkeras.html>

The Fashion-MNIST dataset contains 70 000 grayscale images of 10 different categories of clothing items. The dimensions of each image is 28 x 28 pixels, which makes it suitable for fast training. It’s split into 60 000 training samples and 10 000 test samples. Each image contains one piece of clothing. In all of the experiments, the pixel values were scaled between 0 and 1, and the images were reshaped to fit the input format expected by each model. Aside from that, no extra preprocessing or data augmentation was done. Fashion-MNIST is simple enough to make it easy to test and compare models quickly, but still detailed enough to show clear differences in how well different approaches perform.

To keep experiments reproducible, both NumPy and TensorFlow random seeds were fixed to 42. The dataset loader was implemented to be flexible depending on the model being trained: for instance, it can return either flattened or image-shaped inputs, and optionally one-hot encode labels if needed. Also there’s an option to only grab a smaller subset to quick testing.

III. METHOD

A. Shared concepts and setup

I could discuss an endless amount of concepts around convolution operations, pooling layers, activation functions, gradient descent, loss functions, and regularization techniques, but i assume the reader has an understanding of most of these ideas. I will therefore focus primarily on the main architectural choices and model-specific implementations across the four methods explored: CNN, Autoencoder, GAN, and VAE.

Throughout all models used in this project, a number of common concepts and procedures apply. All image data was preprocessed by normalizing the pixel values to the range $[0, 1]$, and the data was reshaped to match the input expectations of each model. No additional data augmentation or preprocessing was applied. Since we’re working with images, using neural networks that fits for

images is needed. All models were trained using mini-batches of data (typically batch size of 64 or 128), and optimization was performed using either Stochastic Gradient Descent (SGD) or Adam, as described in the lecture notes [4]. Depending on the model, the loss functions included:

- **Categorical Cross-Entropy** for classification tasks:

$$\mathcal{L} = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

- **Mean Squared Error (MSE) or Reconstruction Loss** for generative models.

To improve generalization and reduce overfitting, L2 regularization was applied to relevant layers. This adds a penalty to the loss function based on the weights:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda \sum_j w_j^2$$

The models also used standard activation functions:

- **ReLU (Rectified Linear Unit):**

$$\text{ReLU}(x) = \max(0, x)$$

- **Softmax** for multi-class classification:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

These components formed the basis of all model architectures, and specific configurations are described per method below [1].

B. Convolutional Neural Network (CNN)

CNNs are a natural choice for working with image data like Fashion-MNIST as it makes the explicit assumption that the inputs are images. CNNs are a great fit for working with images, sound, and similar data because they can take advantage of the natural structure in these inputs—like spatial layout in images or time order in audio. Unlike regular dense networks that treat every input the same, CNNs look at local regions and share weights, which helps them learn more efficiently. This makes them much more scalable for large inputs like images, where a fully connected network would need a huge number of parameters. By preserving the relationships between nearby pixels or samples, CNNs are better at recognizing patterns while also reducing the risk of overfitting. I take inspiration from the code displayed in the lecture notes [4] and the model is built up as followed:

Architecture Overview: The CNN used here consists of:

- One 2D convolutional layer with 16 filters of size 3×3 , using ReLU activation and **same** padding
- A 2×2 max pooling layer for spatial downsampling
- A flattening layer to convert the feature maps to a 1D vector
- One dense (fully connected) hidden layer with 64 neurons and ReLU activation
- An output dense layer with 10 units and softmax activation for multi-class classification

A schematic figure of CNN is shown below.

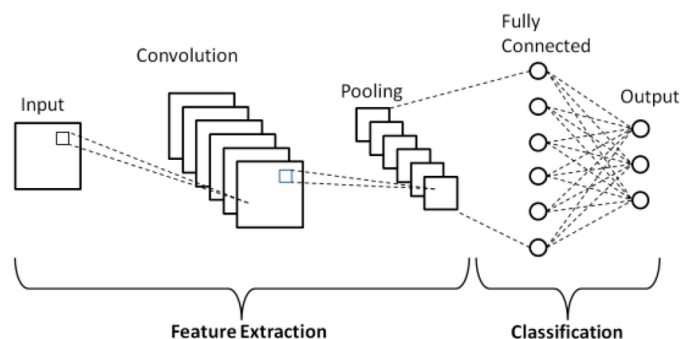


Figure 2. Schematic overview of a CNN. The input passes through convolution and pooling layers to extract features, followed by a fully connected layer and softmax classification. Each output represents a class confidence score in the range $[0, 1]$. Source: https://www.researchgate.net/figure/Schematic-diagram-of-a-basic-convolutional-neural-network-CNN-fig1_336805909

Convolution is a very important concept for CNN. A convolutional layer works by sliding small filters/kernel, usually quite small in size, across the input image (e.g 2×2 or 3×3). Each filter acts like a feature detector, computing a weighted sum over a local region of pixels and producing a new representation called a feature map. The size of the step the filter takes each time is called the stride, a stride of 1 means the filter moves one pixel at a time, while a larger stride reduces the resolution of the output. To control the spatial size of the output, we can apply padding, which adds extra pixels (usually zeros) around the edges of the input image so that the filter can still cover the borders. Padding helps preserve the input dimensions, especially when using small filters. When using the parameter in the layer Conv2D "same", we preserve the dimensions by adding zeros such that it remains the same as input. Together, these choices—filter size, stride, and padding—affect how much the image is compressed and what spatial information is preserved through the layer.

C. Autoencoder (AE)

Autoencoders are a type of neural network designed to learn efficient compressed representations of data. The general idea is that the network tries to replicate the input to produce output matching it. But in doing so, it must compress the input into a smaller latent representation (smaller in size), and then reconstruct the original input from that compressed form. This forces the model to learn the most important features needed to represent the data. This compressed form is often referred to as a bottleneck, as it limits the flow of information and forces the network to prioritize essential features.

The architecture consists of two main parts: the encoder and the decoder. The encoder takes the input image and applies a series of convolutional layers followed by flattening and dense layers to transform it into a low-dimensional latent vector. The decoder then takes this latent vector and tries to reconstruct the original image using transposed convolutional layers. This structure allows the model to learn how to compress and decompress images in a way that retains as much key information as possible.

The network is trained using a reconstruction loss that compares the input to the output. In this project, we used the binary cross-entropy loss:

$$\mathcal{L}_{\text{recon}} = - \sum_{i=1}^N [x_i \log(\hat{x}_i) + (1 - x_i) \log(1 - \hat{x}_i)]$$

where x_i is the original input pixel and $\hat{x}_i$ is the reconstructed pixel. This loss function encourages the output to match the input as closely as possible, pixel by pixel. Below shows the general schematic of an autoencoder.

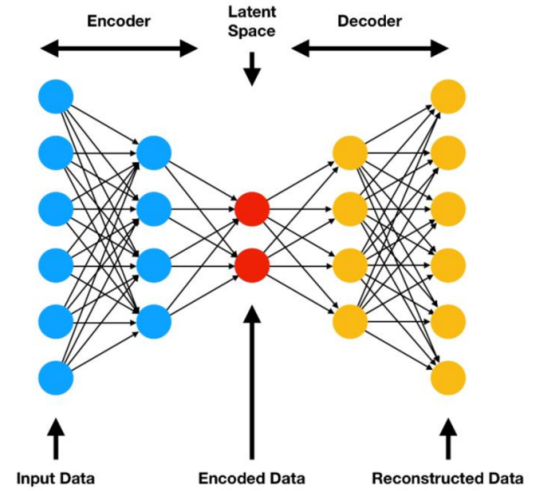


Figure 3. Schematic overview of an autoencoder. The encoder compresses the input data into a low-dimensional latent space, and the decoder attempts to reconstruct the original input from this compressed representation. Source: https://www.researchgate.net/figure/Schematic-of-an-autoencoder-networks-basic-symmetric-architecture_fig5_375222571

In practice, autoencoders can be useful for tasks like denoising, anomaly detection, or dimensionality reduction. For this project, I used them to evaluate how well compressed representations could be used to reconstruct Fashion-MNIST images, and how the model performs compared to more advanced generative methods like GANs and VAEs.

The encoder is built using a series of `Conv2D` layers with `selu` activations, with `MaxPooling2D` to progressively reduce spatial dimensions. If enabled, dropout layers (with rates between 0.2-0.3) are added to introduce regularization and prevent overfitting. The encoder eventually compresses the input into a compact latent feature map of shape $3 \times 3 \times 64$, capturing essential patterns in the image. Code is also mostly pulled from the lecture notes. When needed I made use of AI models like Claude or ChatGPT to fix errors and do some plotting. The SELU activation function used here helps promote self-normalizing properties, which can improve training stability.

The decoder mirrors this structure using `Conv2DTranspose` layers to gradually upsample the latent representation back to the original image resolution. The final output layer applies a sigmoid activation and reshapes the result to match the input size of 28×28 , producing grayscale images with pixel values in $[0, 1]$. The entire autoencoder is trained end-to-end using binary cross-entropy loss and Stochastic Gradient Descent (SGD) with a relatively high learning rate of 1.0. While simple, this setup is seemingly effective for learning meaningful compressed representations and reconstructing the images with reasonable accuracy. This basic autoencoder also serves as a useful baseline

to compare with the more expressive VAE and GAN models later in the report.

D. Variational Autoencoder (VAE)

The Variational Autoencoder (VAE) builds on the idea of a regular autoencoder but adds a probabilistic twist. Instead of compressing each input into a single fixed point in the latent space, the VAE learns to represent it as a distribution. More specifically, the encoder outputs two things: the mean μ and the log-variance $\log \sigma^2$ of a gaussian. This setup allows us to sample different points from that distribution and generate slightly varied outputs, which is great for generative tasks like creating new images. While for autoencoders we could learn latent representation of an image, for VAE we can create new varied samples/images from our learned model with a smooth latent space. While the autoencoding focuses only on minimizing reconstruction error, the VAE also adds a KL divergence term to its loss

To actually sample from the distribution while still allowing gradients to flow through the model during training, we use the so-called reparameterization trick. Instead of drawing samples directly from the learned Gaussian, we generate a standard normal sample $\epsilon \sim \mathcal{N}(0, I)$ and transform it as:

$$z = \mu + \sigma \cdot \epsilon$$

This makes the sampling operation part of the computational graph, which means we can train the whole thing using backpropagation just like any other network.

The loss function has two parts. First, there's the reconstruction loss that tells the model how close the output is to the original input. We used mean squared error (MSE) here:

$$\mathcal{L}_{\text{recon}} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2$$

The second part is the KL divergence, which tells us how different the learned latent distribution $q(z|x)$ is from the prior $p(z)$, usually a standard normal distribution. By keeping this difference small, the VAE makes sure the latent space stays well-behaved and smooth. This means can then sample useful and quite diverse samples from it.

$$D_{KL}[q(z|x)||p(z)] = -\frac{1}{2} \sum_{j=1}^d (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

The final loss is the sum of the reconstruction and KL losses.

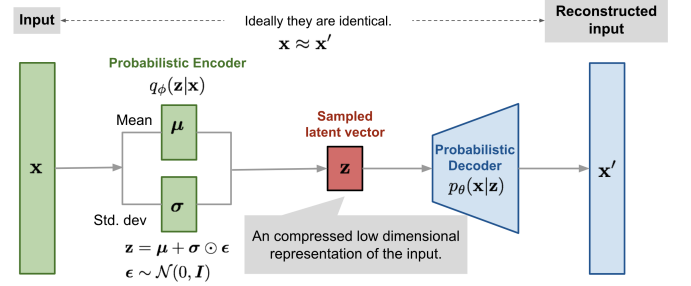


Figure 4. A schematic overview of a Variational Autoencoder. The encoder maps the input x to a distribution parameterized by mean μ and standard deviation σ . A latent vector z is then sampled using the reparameterization trick. The decoder tries to reconstruct the original input from this sampled representation. Ideally, the reconstructed output x' is close to the input.

The overall architecture is fairly standard. The encoder starts with a few convolutional layers using ReLU activations and stride-based downsampling instead of pooling. I added batch normalization and some dropout to improve stability during training. After flattening, the encoder branches into two dense layers—one for the mean μ and one for the log-variance $\log \sigma^2$ —which are used in the reparameterization step to sample a latent vector. The decoder then maps this vector back to a small spatial feature map with a dense layer and uses transposed convolutions to reconstruct the original image. The final sigmoid layer ensures that pixel values stay in the range $[0, 1]$. Code is partly based on the course slides and other reference implementations [4, 6]. I made some adjustments like adding dropout and early stopping, mainly because training took quite a while. The full model was trained on Fashion-MNIST using the Adam optimizer.

E. Generative Adversarial Network (GAN)

GANs are a framework for training two neural networks that compete against each other. The goal is to learn how to generate new images that resemble the training data. The generator takes random noise as input and tries to produce an image that looks like a real example from the dataset. The discriminator is trained to tell the difference between real images from the dataset and fake ones made by the generator. Over time, the generator improves by trying to fool the discriminator, and the discriminator improves by getting better at spotting fakes.

The training process is set up as a minimax game. The discriminator wants to maximize its ability to tell real from fake:

$$\mathcal{L}_D = -[\log D(x) + \log(1 - D(G(z)))]$$

where x is a real image and $G(z)$ is a fake image produced

from noise z . The generator tries to minimize:

$$\mathcal{L}_G = -\log D(G(z))$$

This pushes the generator to make images that the discriminator classifies as real. The combined training objective for both networks can be expressed as:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

Both networks are updated in turn each epoch using their respective losses. The setup often leads to unstable training and is sensitive to hyperparameters. To reduce some of this instability, I used the Adam optimizer with $\beta_1 = 0.5$, which is common for GANs [2].

The generator architecture starts with a dense layer and reshapes the noise vector into a low-resolution feature map. It then uses a combination of upsampling layers followed by convolution to scale the image up to the final 28×28 grayscale format. I replaced `Conv2DTranspose` layers with a resize-then-convolve setup in an attempt to avoid checkerboard artifacts that were visible early in training, an issue mentioned in blog tutorials as well [3].

The discriminator is a standard CNN classifier with two `Conv2D` layers using strides for downsampling. Each layer is followed by LeakyReLU activations and dropout to add some regularization. The output is a single logit that estimates whether the input is real or fake.

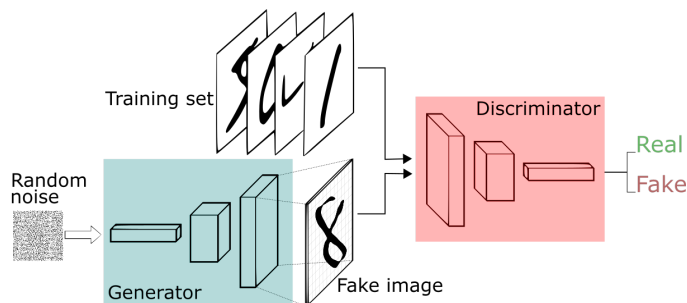


Figure 5. Schematic overview of a GAN. The generator takes a random noise vector and outputs a fake image. The discriminator receives either a real or generated image and tries to classify it correctly. Both models are trained together in a minimax game. Source: <https://www.labellerr.com/blog/what-is-gan-how-does-it-work/>

This setup allows the generator to learn how to produce realistic Fashion-MNIST samples by gradually improving through feedback from the discriminator [5]. While harder to train than simpler models, GANs are powerful for generating new data when tuned and trained correctly. It turned out I had issues with generating realistic images.

IV. RESULTS

A. CNN

To evaluate the CNN performance, I ran a grid search over different learning rates and L2 regularization values. I tested multiple training durations and batch sizes. Two configurations stood out: one using `epochs=10`, `batch_size=32`, and another using `epochs=100`, `batch_size=64`. The former reached a test accuracy of 0.8566 while training relatively quickly. The latter achieved a slightly higher test accuracy of 0.8668 but took significantly longer to train.

The tables below show some results from grid searches done which produces the best two models, one doing well at faster speed and the other best result but longest time spent training.

Table I. Best test accuracy for different training configurations using $\eta = 0.1$.

Epochs	Batch Size	λ	Test Accuracy	Training Time (s)
10	64	$1.00 \cdot 10^{-5}$	0.8380	131.13
10	128	$1.00 \cdot 10^{-5}$	0.8008	122.06
50	64	$1.00 \cdot 10^{-4}$	0.8588	571.86
50	128	$1.00 \cdot 10^{-4}$	0.8108	520.26
100	64	$1.00 \cdot 10^{-4}$	0.8668	1127.24
100	128	$1.00 \cdot 10^{-5}$	0.8502	1004.37

Table II. Best test accuracy for lower batch sizes across different training configurations.

Epochs	Batch Size	λ	Test Accuracy	Training Time (s)
5	16	$1.00 \cdot 10^{-5}$	0.7894	124.87
5	32	$1.00 \cdot 10^{-3}$	0.7018	94.08
10	16	$1.00 \cdot 10^{-4}$	0.8540	226.85
10	32	$1.00 \cdot 10^{-5}$	0.8566	160.71
50	16	$1.00 \cdot 10^{-4}$	0.8652	1028.74
50	32	$1.00 \cdot 10^{-3}$	0.8610	753.53

The heatmaps below show the test accuracy across combinations of η (learning rate) and λ (L2 regularization). As expected, low regularization and moderately high learning rates worked best. Especially for the larger epoch runs, combinations like $\eta = 0.1$ and $\lambda = 1e-4$ consistently gave good results.

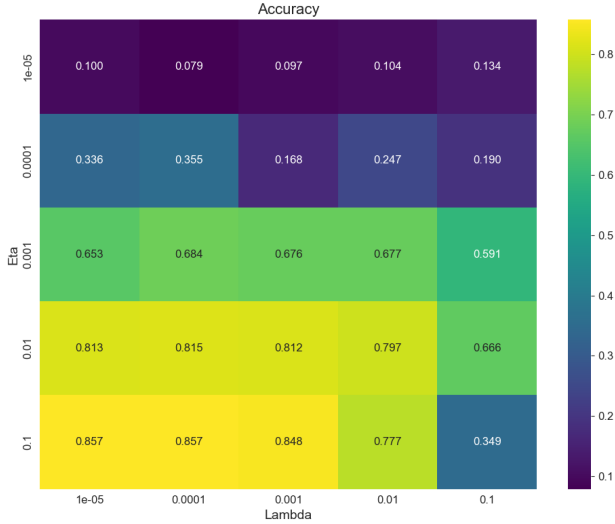


Figure 6. Grid search results with `epochs=10`, `batch_size=32`. Best test accuracy was 0.8566 at $\eta = 0.1$, $\lambda = 1e-5$.

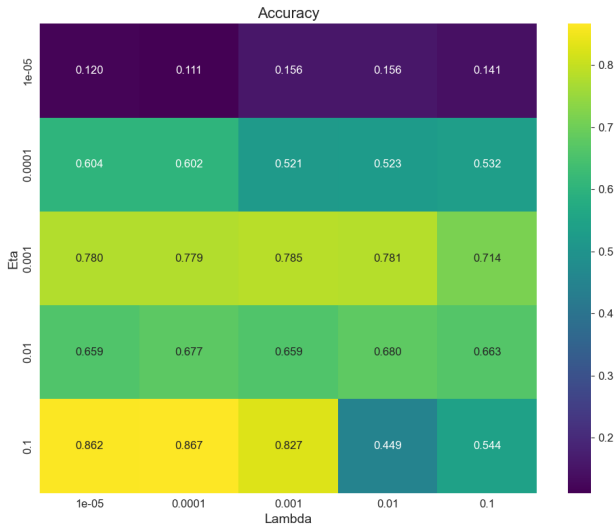


Figure 7. Grid search results with `epochs=100`, `batch_size=64`. Best test accuracy was 0.8668 at $\eta = 0.1$, $\lambda = 1e-4$.

In general, increasing training time improved accuracy, but with diminishing returns. Larger batch sizes slightly reduced performance in most cases. Based on the results, the CNN architecture used here performs well on Fashion-MNIST and is competitive given its simplicity. For most purposes, the quicker 10-epoch setup offers a good balance between training time and accuracy.

B. Autoencoder

The autoencoder code was modified to add dropout. Then tested with and without dropout to study how it

affects reconstruction quality and generalization. Both models used the same architecture and were trained on Fashion-MNIST with binary cross-entropy loss and the SGD optimizer. A grid search was then run over different learning rates ($[0.001, 0.01, 0.1, 1.0]$) and couple different epoch values (10, 50), and the best model in each case was saved and evaluated. Figure 8 and Figure 9 show the training and validation loss curves for the best models found.

The model without dropout achieved a slightly better validation loss than the one with dropout. The best val loss without dropout was 0.2594, compared to 0.2811 with dropout. As seen in the plots, the model with dropout had more unstable validation loss. This suggests that dropout made the model perhaps a bit more robust to overfitting but also slowed convergence. In contrast, the model without dropout trained more smoothly following the training loss quite close. As we look at the reconstructions done by both models (Figure 10 and 11) shows both models captured the general structure of the inputs well. There is some slight blurring in the output, especially for finer patterns and details. As the loss shows for just 50 epochs we can also see that the non-dropout works a bit better when comparing the images.

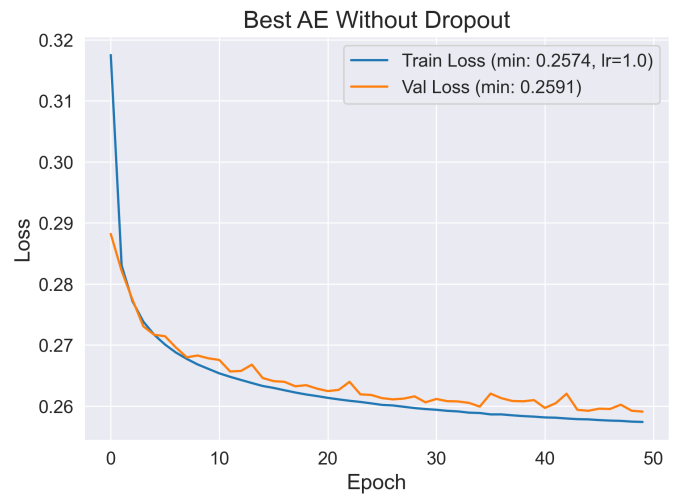


Figure 8. Training and validation loss for the best autoencoder without dropout. Validation and training loss both decrease steadily and reach similar values.

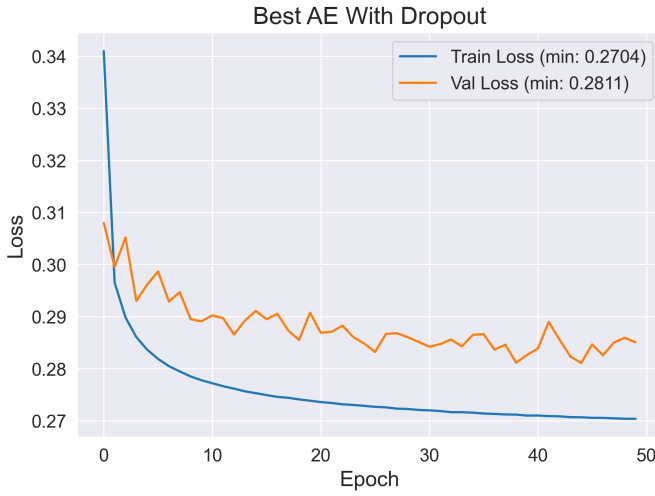


Figure 9. Training and validation loss for the best autoencoder with dropout. The training loss decreases smoothly, but the validation loss shows more fluctuation. Learning rate = 1

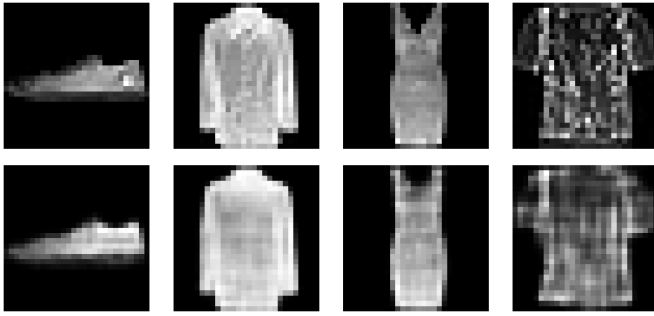


Figure 10. Reconstructions from the best model without dropout. Top row: original images. Bottom row: reconstructed output. Most structure is preserved, though some sharpness is lost. Learning rate = 1

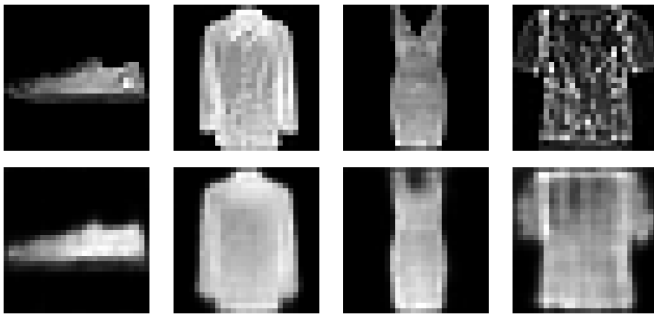


Figure 11. Reconstructions from the best model with dropout. Slightly more blurring is visible in comparison to the model without dropout.

To test whether training longer would help, i ran one extended training with 300 epochs and no dropout. As

shown in Figure 12, the validation loss improved only slightly after 50 epochs and the slope flattened quite fast. I therefore didnt do much further exploration in much longer epoch tests. This indicates that most learning happens early, and training for longer gives diminishing returns for this model. So the basic autoencoder performed reasonably well and gave a strong baseline, especially for a simple setup. It learned to compress and reconstruct the data with decent, though there are limits to the visual sharpness and detail captured by the model.

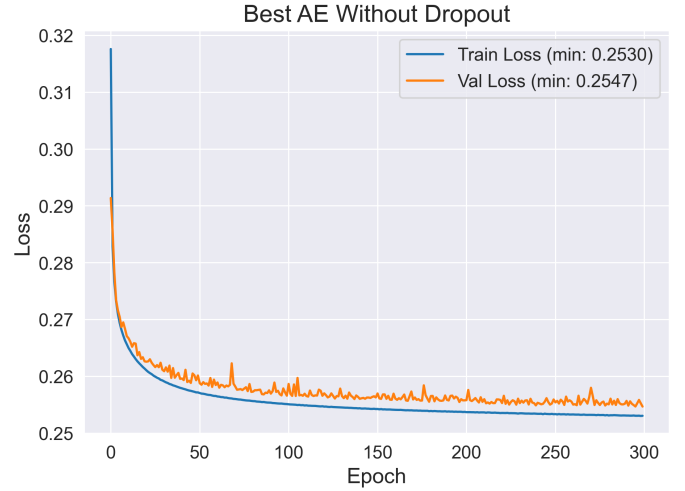


Figure 12. Training and validation loss for an extended 300-epoch run without dropout. The validation loss stabilizes after about 50–100 epochs.

C. Variational Autoencoder

For the training of the VAE, i tried first a latent dim with 2. Then by trying to improve model by adding some more data for the model to make use of i increased the latent space dimension to 10, which aligns with the 10 class categories in the dataset. Then a small grid search was performed over learning rates and epoch counts. The best model used a learning rate of 10^{-3} and trained for 50 epochs with early stopping resulting in a final validation loss of around 6.35. The training and validation losses closely followed each other (Figure 13), indicating stable convergence without major overfitting.

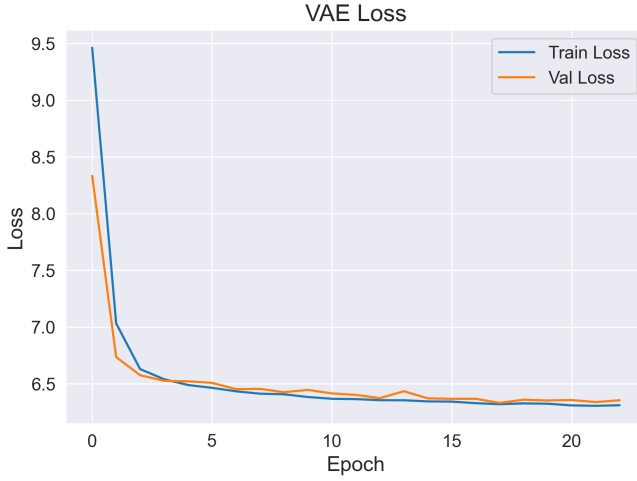


Figure 13. Loss curves for the best VAE model using early stopping. Both training and validation losses decrease steadily and stabilize after around 20 epochs.

I did try grid search here too as show in the table below III. It shows that the model was trained without dropout using a learning rate of 1.0 and 50 epochs, reaching a validation loss of 0.2591. By doing dropout we didnt improve the performance, we just degraded the performance again. The higher learning rates provided better results. While improving epochs did improve results, it didnt really change that much as the slope flattens out at around 20 epochs as show in the figure 13.

Table III. Validation loss and training time for different learning rates and dropout settings.

Dropout	Learning Rate	Epochs	Val Loss	Time (s)
No	1.0	50	0.2591	744
Yes	1.0	50	0.2853	1131
No	0.1	50	0.2695	738
Yes	0.1	50	0.3094	1139
No	0.01	50	0.2924	744
Yes	0.01	50	0.3401	1149

The VAE produced smooth but blurry reconstructions (Figure 14). The general structure of the input was preserved, but fine details were often washed out. This is expected from the regularization enforced by the KL divergence term, which smooths out the latent space. Compared to the standard autoencoder, the VAE traded off some reconstruction fidelity for better generative properties. Even though the generated images were less sharp, the latent space it learns is more structured and useful for sampling or interpolation tasks.

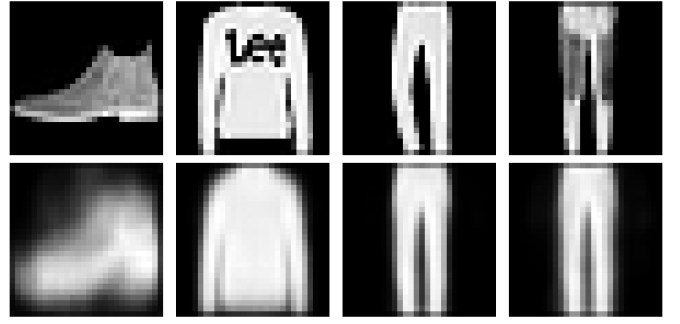


Figure 14. VAE reconstruction results. Top row shows original test images, bottom row shows corresponding reconstructed outputs.

D. GAN Results

Initially i tried a latent dimension equal 64. However that in addition to the generator architecture using `Conv2DTranspose` layers caused severe checkerboard artifacts in the output images. I then changed to trying a dim equal 100. The GAN then was trained for 100 epochs. I looked at several other examples of codes and they used learning rates of around 0.0001-0.0005. I set mine to 0.0002 and attempted a few different batch sizes and set it 0.0002. As well as batch size of 128. To address checkerboard pattern, the architecture was modified to use `UpSampling2D` followed by `Conv2D` layers. This change significantly improved the visual quality by removing the regular grid patterns, as shown in Figure 17. However the resulting attempt at reconstructing the images still got extremely blurry results as show in 15, but at least the clear checkerboard symmetric images was gone.

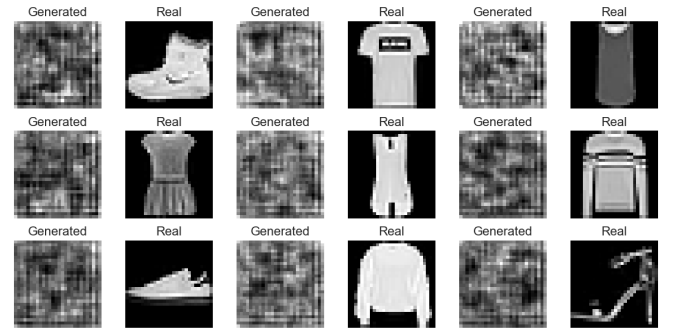


Figure 15. Generated vs real images after 100 epochs of training. The generator uses upsampling followed by convolution, which avoids the checkerboard artifacts seen with transposed convolutions.

Figure 16 shows the loss curves for the generator and discriminator throughout training. The generator loss increases as it learns to produce more convincing images, while the discriminator loss decreases initially before sta-

bilizing. This is seemingly typical GAN training pattern, but the resulting images didnt really present that. Due to the noisy pattern i didnt proceed to try generate new images from random noise as it would've just resulted in similar images.



Figure 16. Training loss for generator and discriminator over 100 epochs.

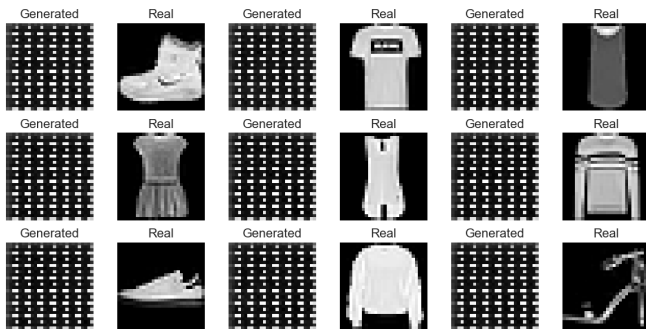


Figure 17. Generated vs real images using the original generator with transposed convolutions. Checkerboard artifacts dominate the generated images.

V. DISCUSSION

A. CNN

The CNN model showed solid performance on the Fashion-MNIST dataset. Even with a relatively simple architecture. By just one convolutional layer, followed by pooling, a hidden dense layer, and a softmax output. It still managed to classify the data with up to 86.7% test accuracy. This is a clear improvement over a basic dense network, and confirms that CNNs are a good fit for image data due to their ability to capture spatial patterns.

The grid search showed that the learning rate had a large impact on accuracy. Models trained with $\eta = 0.1$ consistently outperformed those using lower values. The regularization parameter λ had a smaller but still noticeable effect. A value around $1e-5$ to $1e-4$ gave the best results in most runs. Very high regularization hurt performance, while too little sometimes led to overfitting.

Training for 100 epochs gave the best results, but also came at a higher computational cost. In contrast, training for just 10 epochs gave slightly lower accuracy. However it finishes much faster and doesnt perform much worse. The choice between these setups depends on the goal: if accuracy matters most, training longer is worth it. If speed is more important, shorter runs still works fine. The time ratio is about 7x more for the best model vs the practical model, but only has improvement of about 1 percent in the accuracy. Now one percent might not be worth it here for my use case, but for usage for detecting cancer every small fraction of percent is essential. So the trade off between training time (results of training paramters + model complexity) is an important factor for the programmer to consider.

While not explored here, additional improvements could come from stacking more convolutional layers, using dropout, or experimenting with more advanced optimizers or data augmentation. For this project, the simple setup provided a useful baseline and served as a reference point for comparing the generative models later in the report.

B. Autoencoder

The autoencoder gave decent reconstructions. It did lack the detail and sharpness of the to match the original images. A grid search over learning rate and epochs showed that a high learning rate of 1.0 performed best. This might seem quite high, this might reflect the simple model structure. As maybe the loss space is quite easy to explore. The best validation loss without dropout was around 0.2598, while the version with dropout reached 0.2851. Adding dropout did not improve performance in this case for the training did for 50 epochs. Again this might be due to model being simple so adding dropout just hinders the training. As well as the latent space being quite small, and dataset not being noisy. The reconstruction plots showed that both versions captured the overall shape of clothing items, but without knowing all categories it would be hard to distinguish between all the correct categories. This suggests that the shallow architecture limited the ability to encode detailed spatial information.

Loss curves also showed little improvement after around 40-50 epochs. To test if more training would help, a model was trained for 300 epochs. The result confirmed diminishing return as the validation loss plateaued after a bit. This suggests that the model converges quickly and longer training is unnecessary. Overall, the autoencoder was useful as a baseline generative model, but has clear limitations when used without deeper layers or more complex regularization.

Future work could involve stacking more convolutional layers in the encoder and decoder, adding skip connections, or using different loss functions like mean squared error. Another idea would be to increase the size of the la-

tent space. I would also like try out more visualizations then try out the CNN model on the re-crafted images from the autoencoder to see if it could still classify the images correctly.

C. Variational Autoencoder

The VAE was able to reconstruct Fashion-MNIST images reasonably well, with the best model achieving a validation loss of 0.2591 using a learning rate of 1.0 and 50 training epochs. Models with dropout consistently performed worse, both in terms of reconstruction quality and final validation loss. This indicates that dropout was not beneficial in this context. This aligns with the expectation that VAEs already incorporate a form of regularization via the KL divergence term, which penalizes the complexity in the latent space already. So perhaps it was not needed to test out.

While the reconstructions captured the general shape and category of input images, they lacked sharpness and fine detail. VAEs seemingly is quite known for a bit noisier/blurrier images than GAN which i think stems from the propbalistic nature of the method, but this im unsure of. Furthermore, increasing training time beyond 50 epochs showed diminishing returns, indicating that the model had likely converged.

Future work could involve exploring more expressive decoder architectures or using perceptual or adversarial losses to improve reconstruction quality. It would also be valuable to visualize and interpolate within the learned latent space to evaluate how well the VAE captures the manifold structure of the dataset. Finally, tuning the KL weight (i.e., β -VAE) could offer a better trade-off for the model. Meaning better results. I would like also to try generate some new images using the model.

D. GAN

Despite resolving the checkerboard artifact issue by replacing `Conv2DTranspose` layers with upsampling followed by `Conv2D`, the generator still produced highly noisy and unrealistic images. The outputs lack clear structure, and most samples do not resemble the clothing items in the dataset as presented in the figures. So while the change removed that issue, it was not sufficient to achieve meaningful generation quality of the resulting images.

The loss curves suggest that the adversarial training stabilized, but the generator failed to learn the data distribution effectively. This might be due to a combination of limited training time, perhaps the networks are too shallow and simple. Plus GAN can be quite difficult to train in general.

For future work and improvements there's several things. I could increase depth perhaps of the networks. Additionally visualizations like latent space interpolation

or t-SNE projection of the latent vectors could provide deeper insight into what the GAN is learning. Once the model works properly on reconstructing, i would want to try generate completely new sample images from noise. Lastly, switching to more advanced GAN variants like DCGAN could perhaps improve both convergence and generation performance. More than likely there's a more elementary error in the code i couldnt find when setting up the model or training that results in the images being noisy.

VI. CONCLUSION

In this project, i implemented and compared four types of neural network models on the Fashion-MNIST dataset: a CNN for supervised classification, and three generative models (AE, VAE, GAN) for unsupervised learning and image reconstruction. Each model was trained using TensorFlow/Keras and evaluated on reconstruction quality, training behavior, and sensitivity to different hyperparameters. I tried out dropout and used gridsearch to try find balanced values for learning rates, batch sizes and sometimes epochs to figure out if loss slope flattened out or not.

The CNN performed well as expected, achieving over 86% test accuracy with a relatively simple architecture. The autoencoder was able to reconstruct images with decent fidelity but showed some blurring and lacked sharpness. Adding dropout didn't improve results, likely due to the model's simplicity and the low noise in the data. The VAE extended the AE by learning a probabilistic latent space, producing smoother but slightly blurrier reconstructions. Grid search showed that higher learning rates improved performance, and dropout again didn't help. The GAN proved the most difficult to train. Even as i managed to remove the checkerboard artifacts. The generated images remained noisy and lacked clear structure.

Future work and improvements

For future work, there are several interesting directions to explore. Fixing the GAN by using improved architectures like DCGANs or techniques like gradient penalties could improve image quality. More than likely there exists a more basic error in the architecture or way it's setup thats causing the issues. Visualizing the latent space of the VAE—especially with a low-dimensional latent vector ($\text{dim}=2$) could show insight into how the model organizes different types of clothing which would be fun. We could also try generating completely new images from random noise using both the VAE and GAN once the model is working properly. Another interesting idea to me is to feed reconstructed or generated images into the trained CNN to measure how recognizable they are, or even the new novel images. There's also potential for interpolat-

ing between latent vectors to observe smooth transitions between image types, or even exploring class-conditional generation to make the models more controllable. Overall the CNN while working fine there is still plenty of room for plots and improvement in all three other generative approaches.

-
- [1] Goodfellow, I., Bengio, Y., & Courville, A. [Deep Learning](#). MIT Press (2016).
 - [2] Radford, A., Metz, L., & Chintala, S. [Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks](#). arXiv preprint arXiv:1511.06434 (2015).
 - [3] Odena, A., Dumoulin, V., & Olah, C. [Deconvolution and Checkerboard Artifacts](#). Distill (2016).
 - [4] IN-STK5000 Course Team. Lecture Slides from Week 9, 13, 14, and 16 and corresponding notebooks with code. University of Oslo, Spring 2025.
 - [5] Ahtasham Ahsan. [Fashion-GAN](#). Medium, accessed June 2025.
 - [6] Achraf Louiza. [Image Generation VAE Fashion-MNIST](#). GitHub repository, accessed June 2025.